Sensor Monitoring System (SMS)

Ứng dụng nhúng đa luồng đọc cảm biến qua kernel driver trên ARM Cortex-A9 (QEMU vexpress-a9, Linux 5.15)

Sinh viên	**Nguyễn Ngọc Quang**
MSSV	SE202007
Môn học	LOS 201

Mã nguồn, log đo và video demo nằm trong gói nộp kèm: SE202007_NguyenNgocQuang_Asgn02.tar.gz và SE202007_demo.mp4.

Mục lục

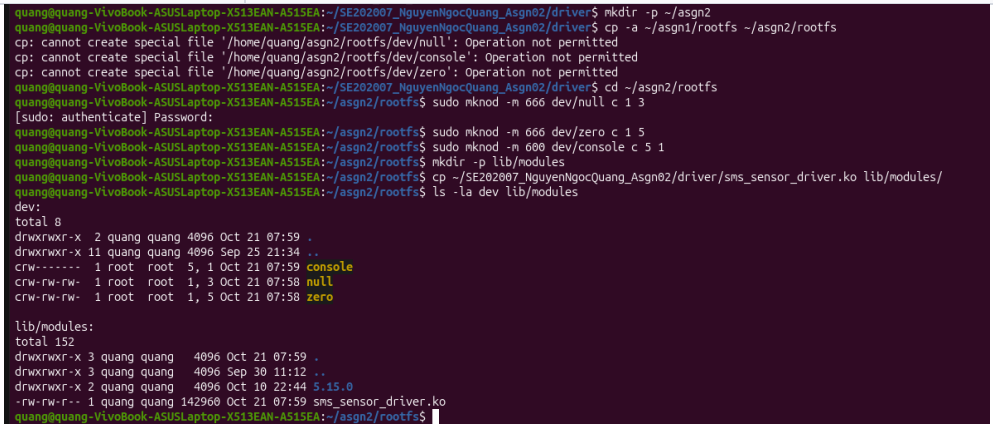
1. Giới thiệu và môi trường thực hiện 3
1.1. Môi trường 3
1.2. Bản đồ yêu cầu → bằng chứng 3
2. Thiết kế kiến trúc 5
2.1. Kiến trúc tổng thể 5
2.2. Các luồng và vai trò 5
2.3. Cấu trúc dữ liệu và các quyết định chính trong luồng Sensor 6
2.4. Kernel driver sms_sensor_driver.ko 6
2.5. State machine của ứng dụng 7
2.6. Giao thức tắt máy (graceful shutdown) 7
2.7. Các quyết định thiết kế quan trọng 8
3. Phân tích đồng bộ hóa giữa các thread 9
3.1. Dữ liệu dùng chung và cơ chế bảo vệ 9
3.2. Thứ tự khóa và chứng minh không có deadlock 9
3.3. Các tình huống thời gian / đua dữ liệu tiềm ẩn và cách xử lý 10
4. Kết quả đo lường 12
4.1. Bảng tổng hợp lần chạy 60 giây 12
4.2. Số alert quan sát so với kỳ vọng lý thuyết 12
4.3. Tích hợp driver: /proc/sms_stats, ioctl, lỗi và file descriptor 13
4.4. Memory footprint 14
4.5. Đo jitter của Sensor Thread (Task 4) 15
5. Debug và profiling 18
5.1. Valgrind — memcheck 18
5.2. GDB — Kịch bản A: quan sát shared_buffer khi Sensor đang ghi 18
5.3. strace — mẫu system call 19
5.4. perf — điểm nóng CPU 21
6. Reflection — nhìn lại toàn khóa 23
6.1. Kỹ năng khó nhất và cách tôi vượt qua 23
6.2. Nếu làm lại, tôi sẽ thiết kế khác 23
6.3. Kỹ năng sẽ dùng nhiều nhất trong thực tế 23
6.4. Một cải tiến kỹ thuật cụ thể nếu có thêm thời gian 24
7. Tài liệu tham khảo và công cụ 25
Phụ lục 26
A. Cấu trúc gói nộp 26
B. Lệnh tái hiện 26
C. Trích đoạn mã chính 26

1. Giới thiệu và môi trường thực hiện

Báo cáo mô tả thiết kế, triển khai và kết quả đo của **Sensor Monitoring System (SMS)**: ứng dụng nhúng đa luồng trên ARMv7 (QEMU `vexpress-a9`) đọc nhiệt độ/độ ẩm từ kernel driver tự viết `sms_sensor_driver.ko`. Mọi số liệu lấy từ log thật trong `logs/`; ảnh chụp màn hình là phiên làm việc của tôi ngày 21/10/2026.

1.1. Môi trường

Hạng mục	Giá trị
Máy phát triển	Ubuntu (laptop x86_64); cross-compiler `arm-linux-gnueabihf-gcc` 12.5.0
Máy đích	QEMU `vexpress-a9`, 2 vCPU, 512 MB RAM, ARMv7 (Cortex-A9)
Kernel	Linux 5.15.0 tự build (`#3 SMP PREEMPT`); `CONFIG_HZ=1000`, `CONFIG_HIGH_RES_TIMERS`, `CONFIG_PREEMPT`, `CONFIG_POSIX_MQUEUE`
Root filesystem	BusyBox + initramfs (`rdinit=/sbin/init`), file `output/initramfs_asgn2.cpio.gz`; `sms_app` build tĩnh (static) cho ARM
Công cụ phân tích	Valgrind 3.26.0, ThreadSanitizer (gcc), GDB, strace, perf, `/usr/bin/time -v` (trên bản x86_64 `sms_app_x86`, chế độ mô phỏng `-D sim`); `/proc` và đo jitter nội bộ (trên QEMU với driver thật)
Ngôn ngữ / cờ biên dịch	C (gnu11), `-O2 -g -fno-omit-frame-pointer -Wall -Wextra` — build sạch, không warning



Hình 1 — Tạo rootfs ASGN2 từ ASGN1: `mknod` các node `dev/`, chép `.ko` vào `lib/modules/` (142 960 B).



Hình 2 — Kernel có `CONFIG_DEVTMPFS=y` nhưng chưa tự mount, nên thêm `mount -t devtmpfs none /dev` vào `init` để `/dev/sms_sensor` xuất hiện.

1.2. Bản đồ yêu cầu → bằng chứng

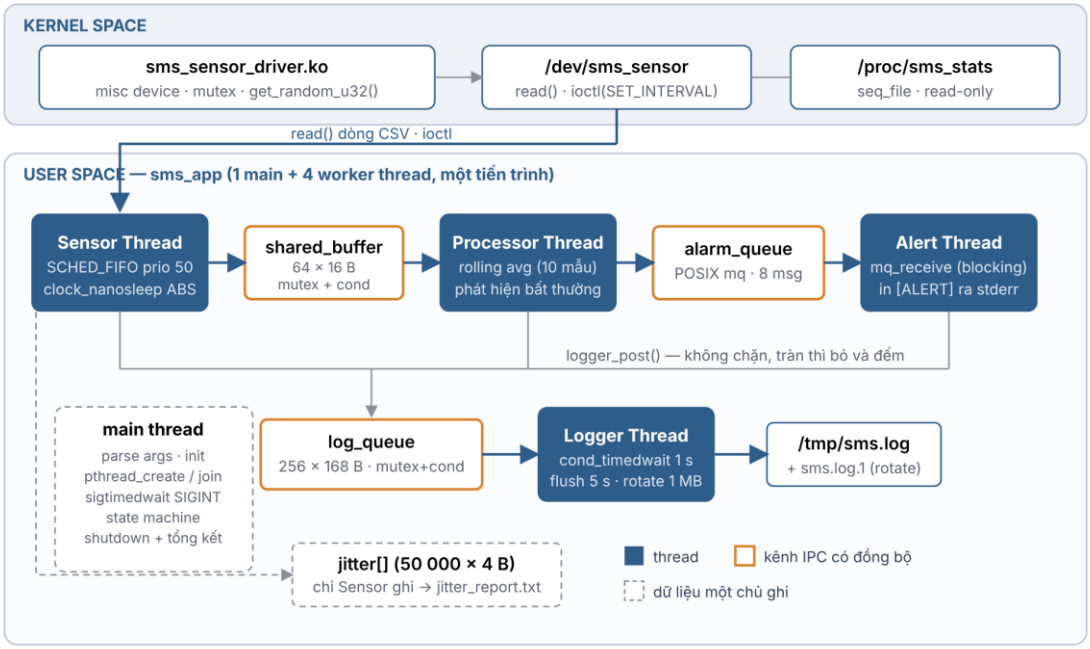
Task	Yêu cầu chính	Bằng chứng (mục báo cáo / file log)
T1	Driver, 4 thread, circular buffer, mq, graceful shutdown	§2, §3, §4.1; `sms_sample_run.txt`, `sms.log`, `run_exit_code.txt`
T2	Valgrind 0 lost; footprint 3 thời điểm; maps; ioctl/EAGAIN/EINTR; fd; `/proc/sms_stats`	§4.2–§4.4, §5.1; `valgrind_sms.txt`, `memory_T5/T30/T55.txt`, `proc_maps.txt`, `fd_table_T55.txt`, `proc_sms_stats_*.txt`
T3	GDB (kịch bản A), strace top-5, perf hotspot + đề xuất	§5.2–§5.4; `gdb_sms.txt`, `strace_sms.txt`, `perf_sms.txt`

Task	Yêu cầu chính	Bằng chứng (mục báo cáo / file log)
T4	SCHED_FIFO, `clock_nanosleep`, `mlockall`, ≥1000 mẫu jitter, FIFO vs OTHER	§2.3, §4.5; `jitter_report.txt`, `jitter_main_run.txt`
T5	Báo cáo, sơ đồ, demo, reflection	toàn bộ tài liệu; §6; `demo/SE202007_demo.mp4`

2. Thiết kế kiến trúc

2.1. Kiến trúc tổng thể

SMS gồm một kernel module và một tiến trình userspace có **5 luồng**: luồng *main* điều khiển vòng đời và bốn luồng làm việc (Sensor, Processor, Alert, Logger). Dữ liệu chảy một chiều từ cảm biến đến file log; các thành phần chỉ nói chuyện với nhau qua ba kênh có đồng bộ rõ ràng: `shared_buffer`, `alarm_queue` (POSIX mq) và `log_queue`.



Hình 3 — Kiến trúc hệ thống SMS: kernel module, 4 thread làm việc, các kênh IPC và luồng dữ liệu.

2.2. Các luồng và vai trò

Thread	Chạy khi nào / chờ gì	Nhiệm vụ	Lịch / ghi chú
Sensor	Mỗi chu kỳ `-i` ms (mặc định 500) qua `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME)`	Đo jitter; `read()` một dòng CSV từ `/dev/sms_sensor`; parse; `cbuf_push` vào `shared_buffer` (đầy → log WARN, bỏ mẫu, đếm overflow)	SCHED_FIFO prio 50; thread duy nhất mở khóa SIGUSR1; xử lý EAGAIN/EINTR/EOF
Processor	Chặn trên condition variable của `shared_buffer` đến khi có mẫu	Trung bình trượt 10 mẫu (nhiệt độ, độ ẩm); phát `TEMP_HIGH` (>40 °C), `HUMID_HIGH` (>80 %), `TEMP_SPIKE` (|ΔT|>5 °C); gửi cảnh báo vào mq; gửi mẫu đã xử lý sang Logger	SCHED_OTHER; `mq_send` không chặn
Alert	Chặn trong `mq_receive`	In `[ALERT][ts] TYPE: … VALUE: … THRESHOLD: …` ra stderr, đếm theo loại, chuyển sang Logger	Thoát khi nhận thông điệp lính canh `ALERT_STOP`
Logger	Chặn trên `log_queue` với timeout 1 s	Là *writer duy nhất* của `/tmp/sms.log`: thêm timestamp đầy đủ, `fflush` mỗi 5 s, xoay file khi ≥ 1 MB (`sms.log` → `sms.log.1`)	Thoát khi `log_queue` đã đóng và rỗng
main	Vòng `sigtimedwait` 200 ms	Phân tích tham số, khởi tạo, tạo thread, kiểm tra hết giờ `-d`, điều phối tắt máy, in tổng kết	SIGINT/SIGTERM chỉ main nhận (các thread khác bị chặn)

2.3. Cấu trúc dữ liệu và các quyết định chính trong luồng Sensor

- **Circular buffer tổng quát** `cbuf_t` (chứa bản sao các phần tử kích thước cố định): cấp phát đúng một lần bằng `calloc` lúc khởi tạo; `head/tail/size`, bộ đếm `total_produced/total_consumed/overflow_count` và các getter. Cùng một cài đặt dùng cho `shared_buffer` (64 × `SensorData` 16 B) và `log_queue` (256 × `LogMsg` 168 B).
- **Hợp đồng không chặn cho producer**: `cbuf_push`, `mq_send` (`O_NONBLOCK`) và `logger_post` không bao giờ ngủ; hết chỗ thì bỏ phần tử và tăng bộ đếm. Nhờ vậy luồng thời gian thực không bị kẹt sau một consumer chậm (xem §3.3).
- **Hẹn giờ tuyệt đối**: `next += period; clock_nanosleep(…, TIMER_ABSTIME, &next)` — sai số không tích lũy như `usleep(period)`. Jitter = `actual − expected` của từng chu kỳ, lưu trong mảng 50 000 phần tử (một chủ ghi).
- **Giảm page fault**: `mlockall(MCL_CURRENT | MCL_FUTURE)` ngay đầu `main`; stack mỗi thread 256 KB, mảng jitter và hai buffer cấp phát trước khi thread chạy.
- **Guard band cho driver** (phát hiện qua đo đạc, §4.5): driver chặn `read()` nếu đến sớm hơn `interval_ms` kể từ lần đọc trước. Ứng dụng đã tự hẹn giờ chính xác nên chỉ đặt ngưỡng của driver ở **90 % chu kỳ** (`SMS_APP_DRV_GUARD_DIV = 10`): với `-i 500` driver nhận `interval_ms = 450` (thấy trong `/proc/sms_stats`).
- **Chế độ mô phỏng** `-D sim`: sinh dòng CSV cùng định dạng ngay trong ứng dụng để chạy Valgrind/perf/GDB trên PC x86 nơi không có driver; đường dẫn thiết bị luôn lấy từ `#define` hoặc tham số, không hard-code.

2.4. Kernel driver `sms_sensor_driver.ko`

- **Đăng ký**: `misc_register()` (minor động) → `/dev/sms_sensor` mode 0666; `proc_create()` + `seq_file` → `/proc/sms_stats`.
- **read()**: giữ mutex; chưa đủ `interval_ms` thì `msleep_interruptible` (hoặc `-EAGAIN` nếu `O_NONBLOCK`); sinh T 15–45 °C, H 30–90 % (lưu ×10) rồi `copy_to_user` chuỗi CSV.
- **ioctl** `SMS_SET_INTERVAL`: ngoài [MIN, MAX] → `-EINVAL`; mã lạ → `-ENOTTY`; mặc định 100 ms.
- **Dọn dẹp**: `remove_proc_entry` + `misc_deregister` ở `module_exit`.

Biên dịch trên ARM32. Lần build đầu lỗi `modpost: "__aeabi_uldivmod" undefined` do phép chia `u64 / 1000` trong `/proc`; kernel không liên kết libgcc. Sửa bằng `div_u64()` (`<linux/math64.h>`), build lại thành công (`.ko` 140 KB, ELF ARM EABI5, còn debug_info). Cảnh báo lệch phiên bản gcc (12.5.0 vs 15.2.0) không ảnh hưởng.

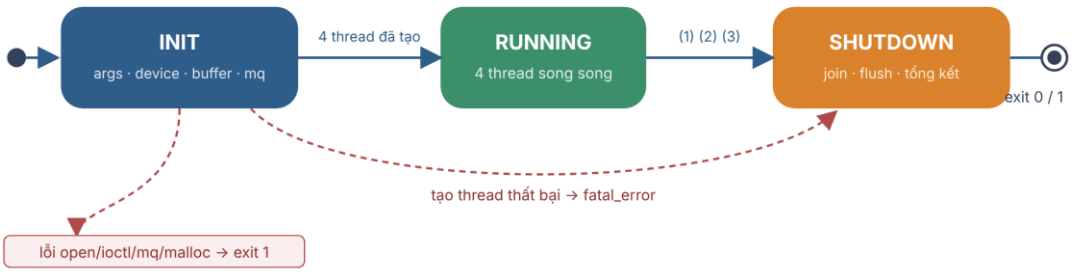
```
make -C /home/quang/embedded_lab1/kernel/linux-5.15 M=/home/quang/SE202007_NguyenNgocQuang_Asgn02/driver ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: arm-linux-gnueabihf-gcc-12 (Ubuntu 12.5.0-9ubuntu1) 12.5.0
  You are using:           arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
ERROR: modpost: "__aeabi_uldivmod" [/home/quang/SE202007_NguyenNgocQuang_Asgn02/driver/sms_sensor_driver.ko] undefined!
make[2]: *** [scripts/Makefile.modpost:134: /home/quang/SE202007_NguyenNgocQuang_Asgn02/driver/Module.symvers] Error 1
make[2]: *** Deleting file '/home/quang/SE202007_NguyenNgocQuang_Asgn02/driver/Module.symvers'
make[1]: *** [Makefile:1783: modules] Error 2
make: *** [Makefile:12: all] Error 2
```

Hình 4 — Lỗi `__aeabi_uldivmod` khi modpost.

```
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/SE202007_NguyenNgocQuang_Asgn02/driver$ grep -n "div_u64\|math64" sms_sensor_driver.c
22:#include <linux/math64.h>
158:    seq_printf(m, "uptime_sec: %llu\n", div_u64(sms_now_ms() - sms.start_ms, 1000));
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/SE202007_NguyenNgocQuang_Asgn02/driver$ make
make -C /home/quang/embedded_lab1/kernel/linux-5.15 M=/home/quang/SE202007_NguyenNgocQuang_Asgn02/driver ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: arm-linux-gnueabihf-gcc-12 (Ubuntu 12.5.0-9ubuntu1) 12.5.0
  You are using:           arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/SE202007_NguyenNgocQuang_Asgn02/driver$ ls -lh sms_sensor_driver.ko
-rw-rw-r-- 1 quang quang 140K Oct 21 07:55 sms_sensor_driver.ko
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/SE202007_NguyenNgocQuang_Asgn02/driver$ file sms_sensor_driver.ko
sms_sensor_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=5aeaea4abca1ed0655ba5997f74b34c199bf1848, with debug_info, not stripped
```

Hình 5 — Sau khi dùng `div_u64`: build xong, `file` xác nhận module ARM.

2.5. State machine của ứng dụng



Hình 6 — State machine INIT → RUNNING → SHUTDOWN. Điều kiện (1) SIGINT/SIGTERM, (2) hết thời gian `-d`, (3) `fatal_error` (10 lần đọc cảm biến lỗi liên tiếp, `mq_receive` lỗi, không mở lại được log).

Chuyển trạng thái	Điều kiện	Hành động
INIT → RUNNING	Mở device, `ioctl`, tạo buffer, mq; 4 thread tạo xong	Ghi log `[STATE] INIT -> RUNNING`; main vào vòng `sigtimedwait`
INIT → thoát	Tham số sai (exit 2); thiếu driver / ioctl / mq / bộ nhớ (exit 1)	`free_resources()` an toàn với tài nguyên khởi tạo dở; in thông báo gợi ý (ví dụ "is sms_sensor_driver.ko loaded?")
RUNNING → SHUTDOWN	SIGINT/SIGTERM, hết `-d` giây, hoặc `fatal_error`	Đặt `running = 0`, chạy giao thức tắt máy (§2.6)
SHUTDOWN → thoát	Mọi thread đã join	In tổng kết, ghi báo cáo jitter, giải phóng tài nguyên; exit 0 (bình thường) hoặc 1 (có `fatal_error`)

2.6. Giao thức tắt máy (graceful shutdown)

Thứ tự dừng đi theo chiều dòng dữ liệu, mỗi bước chờ `pthread_join` xong rồi mới sang bước sau, vì vậy không mẫu nào đang trên đường đi bị mất:

1. **Dừng producer**: `running = 0`; `pthread_kill(sensor, SIGUSR1)` làm `clock_nanosleep`/`read` trả `EINTR` (handler rỗng, không có `SA_RESTART`); `join` Sensor.
2. **Xả Processor**: `cbuf_close(sensor_buf)` — consumer xử lý nốt phần còn lại rồi nhận `CBUF_CLOSED`; `join` Processor.
3. **Đánh thức Alert**: gửi thông điệp `ALERT_STOP` (priority 31) vào mq (thử lại đến ~1 s nếu mq đầy); `join` Alert.
4. **Chốt log**: đẩy 3 dòng `[SUMMARY]` rồi `cbuf_close(log_buf)`; Logger xả hết, `fflush` lần cuối, thoát; `join` Logger.
5. **Dọn**: in tổng kết ra terminal, ghi `jitter_report`, `mq_close`/`mq_unlink`, `close(dev_fd)`, `fclose(log)`, `cbuf_destroy`, `free`.

2.7. Các quyết định thiết kế quan trọng

Câu hỏi	Quyết định và lý do
Circular buffer thay vì linked list?	Số phần tử và kích thước cố định: cấp phát heap *một lần*, push/pop O(1) xác định (hợp luồng RT), bộ nhớ liền kề, tràn là sự kiện đếm được (`overflow_count`) thay vì RAM tăng vô hạn.
POSIX mq cho alarm thay vì shared memory?	Alarm là thông điệp rời rạc, có thứ tự (24 B); `mq_receive` chặn tự nhiên, kernel lo đồng bộ và priority. Shared memory chỉ cần khi chia sẻ khối lớn giữa *tiến trình*.

Câu hỏi	Quyết định và lý do
Logger thread riêng?	Ghi file có thể chặn, không được làm trễ Sensor; Logger là writer duy nhất nên không cần mutex file; gom flush mỗi 5 s; rotate tại một chỗ.
Producer không bao giờ chặn?	Chấp nhận mất mẫu có đếm còn hơn làm Sensor ngủ (jitter, nguy cơ vòng chờ). Mất mát được báo ở phần tổng kết.
Đồng hồ	`CLOCK_MONOTONIC` cho hẹn giờ, jitter và `cond_timedwait`; `CLOCK_REALTIME` chỉ cho timestamp log.

3. Phân tích đồng bộ hóa giữa các thread

3.1. Dữ liệu dùng chung và cơ chế bảo vệ

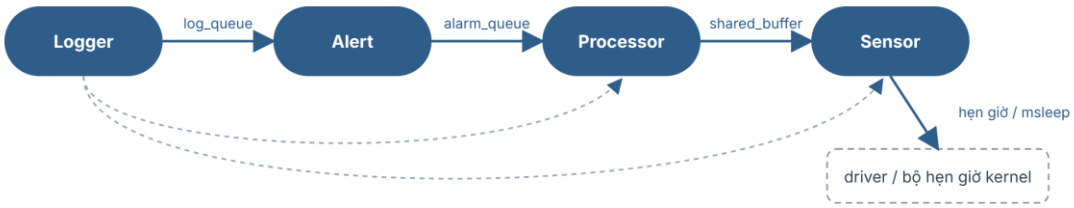
Dữ liệu	Ai truy cập	Cơ chế bảo vệ	Lý do / ghi chú
`shared_buffer` (`head, tail, size`, bộ đếm, mảng dữ liệu)	Sensor (push); Processor (pop); main (getter khi tổng kết)	`pthread_mutex_t` riêng của buffer + `pthread_cond_t not_empty`	Mọi trường đọc/ghi trong vùng găng của mutex; `cond_wait` đặt trong vòng `while` chống spurious wake-up
`log_queue`	Sensor, Processor, Alert, main (push); Logger (pop)	Mutex + cond riêng, cùng cài đặt `cbuf_t`	Nhiều producer → một consumer; push không chặn
`alarm_queue`	Processor (send); Alert, main (send `ALERT_STOP`)	Kernel POSIX mq	Đồng bộ nằm trong kernel; gửi bằng descriptor `O_NONBLOCK`, nhận bằng descriptor chặn
Bộ đếm thống kê (`samples_read`, `alerts[]`, `alarms_dropped`, `overruns`, …)	Nhiều thread ghi, main đọc	`atomic_ulong` (C11), `fetch_add`	Chỉ cần tính nguyên tử, không có bất biến liên quan giữa các bộ đếm
`running`, `fatal_error`	Mọi thread đọc; main/Sensor/Logger/Alert ghi	`atomic_int` (seq_cst)	Cờ điều khiển một chiều; không cần mutex
Mảng `jitter_us[]`, `jitter_count`	Sensor ghi; main đọc sau khi Sensor đã kết thúc	*Một chủ ghi* + `pthread_join` (quan hệ happens-before)	Không khóa trong vòng thời gian thực
`log_fp`, `log_bytes`	Chỉ Logger khi đang chạy; main sau khi join	Sở hữu duy nhất (single-owner)	Không cần khóa file
Trạng thái driver (`read_count`, `last_*`, `interval_ms`)	Mọi process mở device; `/proc`	`struct mutex` của driver	Cả `read`, `ioctl` và `/proc/sms_stats` dùng chung một mutex

3.2. Thứ tự khóa và chứng minh không có deadlock

Deadlock chỉ xảy ra khi đồng thời có bốn điều kiện Coffman: loại trừ tương hỗ, giữ-và-chờ, không thu hồi, và chờ vòng. Thiết kế loại bỏ hai điều kiện, đủ để chứng minh:

1. **Không có giữ-và-chờ trên mutex.** Có ba mutex ứng dụng, thuộc ba đối tượng khác nhau: `sensor_buf.lock`, `log_buf.lock` (của `cbuf_t`) và mutex của driver (trong kernel). Trong `circular_buffer.c` mọi hàm lấy **đúng một** mutex, và **không gọi hàm nào khác có lấy khóa khi đang giữ nó**: bên trong vùng găng chỉ có `memcpy`, cập nhật chỉ số và `pthread_cond_signal`. Hàm `logger_post` (lấy `log_buf.lock`) chỉ được gọi *bên ngoài* các vùng găng của `sensor_buf`. Vì không thread nào giữ hai khóa cùng lúc nên không có thứ tự lấy khóa nào có thể đảo ngược; nói cách khác, "lock order" chỉ có một bậc và đồ thị phân cấp khóa không có cạnh.
2. **`pthread_cond_wait` nhả mutex khi ngủ**, nên thread đang chờ dữ liệu *không giữ* khóa mà producer cần.
3. **Producer không bao giờ chặn chờ consumer.** `cbuf_push` trả `CBUF_FULL` ngay; `mq_send` dùng `O_NONBLOCK`; `logger_post` chỉ gọi `cbuf_push`. Do đó trong đồ thị *wait-for* (thread A chờ thread B) chỉ tồn tại các cạnh **consumer → producer** — hình 3 — và đồ thị này là DAG (không có chu trình).

A → B nghĩa là "A có thể ngủ chờ B". Nét liền: chờ trực tiếp; nét đứt: chờ gián tiếp.



Hình 7 — Đồ thị wait-for của SMS. Mọi cạnh đi từ consumer sang producer rồi kết thúc ở bộ hẹn giờ/driver trong kernel; không có chu trình nên không thể có deadlock. (Logger còn chờ gián tiếp Processor và Sensor vì cả hai cùng đẩy log vào `log_queue` — nét đứt; main chỉ *đánh thức* consumer bằng `cbuf_close` chứ không bị chờ.)

Để làm rõ bằng ngôn ngữ Resource Allocation Graph: có 3 nút tài nguyên (mutex) mỗi nút loại "1 thể hiện", 5 nút tiến trình/thread; mỗi thread có *tối đa một cạnh cấp phát* (tài nguyên → thread) tại một thời điểm và không có cạnh yêu cầu nào xuất phát từ thread đang giữ một tài nguyên. Một chu trình cần ít nhất hai thread, mỗi thread vừa giữ vừa yêu cầu — điều không bao giờ xảy ra ở đây.

Kiểm chứng thực nghiệm. (i) Chạy 60 s trên QEMU với 2 vCPU: không treo, thoát mã 0, mọi thread kết thúc (§4.1). (ii) Bản host biên dịch với **ThreadSanitizer** (`-fsanitize=thread`), chạy 10 s, chu kỳ 20 ms, 500 mẫu: **0 cảnh báo** data race hay lock-order inversion. (iii) Valgrind (§5.1): 0 lỗi. Các thử nghiệm không thay thế chứng minh ở trên nhưng khớp với nó.

3.3. Các tình huống thời gian / đua dữ liệu tiềm ẩn và cách xử lý

Tình huống	Hậu quả nếu bỏ qua	Cách xử lý trong code
Alert thread chặn mãi trong `mq_receive` khi tắt máy (lost wake-up)	`pthread_join(alert)` không bao giờ trở về, chương trình treo sau Ctrl+C	Thông điệp lính canh ALERT_STOP gửi sau khi Processor đã join (không còn cảnh báo mới). Dùng priority 31 để lính canh không bị kẹt sau hàng đợi đầy; đánh đổi là `mq_receive` trả thông điệp priority cao trước nên cảnh báo còn tồn trong hàng đợi (nếu có) sẽ không được xử lý. Alert xử lý ngay nên hàng đợi gần như luôn rỗng: lần chạy 60 s có 130 cảnh báo được đếm = 130 dòng `[ALERT]` trong log, 0 bị bỏ
Tín hiệu SIGUSR1 đến trước khi Sensor chặn	Handler chạy sớm, sau đó `clock_nanosleep` ngủ trọn một chu kỳ; tệ nhất chậm một chu kỳ khi thoát	`running = 0` được đặt *trước* khi gửi tín hiệu; sau mỗi EINTR/lần tỉnh Sensor kiểm tra lại `running`. Độ trễ tắt máy bị chặn trên bởi một chu kỳ (≤ 0,5 s với cấu hình mặc định) — chấp nhận được và đã quan sát thấy tắt sạch.
Mất mẫu khi tắt máy	Mẫu còn trong `shared_buffer` bị bỏ	`cbuf_close` chỉ chặn push mới; `cbuf_pop` vẫn trả phần tử còn lại và chỉ trả CBUF_CLOSED khi rỗng. Kết quả: `produced = consumed` (119 = 119)
Tràn buffer làm chặn luồng RT	Sensor ngủ chờ chỗ trống → jitter, có thể tạo vòng chờ	Producer luôn không chặn, tràn thì bỏ và đếm (`overflow = 0` trong mọi lần chạy)
Byte padding chưa khởi tạo gửi qua mq	`AlarmMsg` có padding; Valgrind sẽ báo "syscall param mq_send points to uninitialised byte(s)", và dữ liệu rác lọt vào kernel	`memset(&m, 0, sizeof m)` trước khi điền (`raise_alarm`, `alert_send_stop`)

Tình huống	Hậu quả nếu bỏ qua	Cách xử lý trong code
Đồng hồ treo tường nhảy	`cond_timedwait` của Logger ngủ sai thời gian	Condition variable dùng `CLOCK_MONOTONIC`
Chặn trong driver làm méo chu kỳ (phát hiện bằng đo, §4.5)	Jitter trung bình ~11 ms trên chu kỳ 20 ms, mất ~8 % chu kỳ	Ứng dụng đặt ngưỡng của driver ở 90 % chu kỳ (guard band) → jitter 157,6 µs

4. Kết quả đo lường

Các số trong mục này lấy từ lần chạy chuẩn 60 s trên QEMU (driver thật, `sms_app -v -d 60`, mặc định chu kỳ 500 ms, SCHED_FIFO 50) do script `asgn2_collect.sh` thu thập, và từ hai lần chạy jitter 25 s dưới tải. File gốc nằm trong `logs/`.

4.1. Bảng tổng hợp lần chạy 60 giây

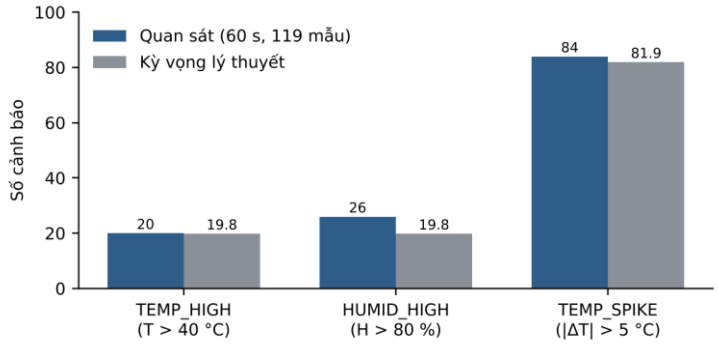
Chỉ tiêu	Kết quả	Ghi chú
Thời gian chạy / mã thoát	60,0 s / 0	`run_exit_code.txt`; tắt do hết `-d 60`
Số sample đọc / đã xử lý	119 / 119	Mẫu đầu tiên ở T+0,5 s; mẫu thứ 120 trùng đúng lúc kết thúc nên không còn
Buffer: produced / consumed	119 / 119	Không mất mẫu khi tắt máy (xả hết trước khi thoát)
Buffer overflow	0 (0 %)	Chiều sâu buffer ghi lúc in mẫu: luôn `buf=0` (consumer theo kịp)
Alert TEMP_HIGH (T > 40 °C)	20	Tổng 130; dropped (mq đầy) = 0; số dòng `[ALERT]` trong `sms.log` = 130
Alert HUMID_HIGH (H > 80 %)	26	
Alert TEMP_SPIKE (|ΔT| > 5 °C)	84	
Lỗi đọc cảm biến / chu kỳ bị lỡ	0 / 0	Không có EAGAIN hay EINTR bất thường
Log bị mất (log_queue đầy)	0	Dung lượng 256 mục, Logger rỗng hàng đợi mỗi ≤ 1 s
Jitter (chu kỳ 500 ms, 119 mẫu)	166 / 452,1 / 629 µs	Min / Avg / Max; P99 = 592 µs (`jitter_main_run.txt`)

`sms.log` có dòng từ cả bốn thread (đúng tiêu chí "log file có entry từ tất cả thread"): `[INFO]/[STATE]/[SUMMARY]` từ main, `[INFO] sensor thread scheduling` từ Sensor, `[SAMPLE]` từ Processor, `[ALERT]` từ Alert, và chính Logger là bên ghi file. Mẫu trích đầu và cuối file:

```
2026-10-21 02:42:09.436 [INFO] SMS start: interval=500ms temp_thr=40.0 hum_thr=80.0 duration=60s
dev=/dev/sms_sensor
2026-10-21 02:42:09.582 [STATE] INIT -> RUNNING
2026-10-21 02:42:09.602 [INFO] sensor thread scheduling: SCHED_FIFO prio=50
2026-10-21 02:42:11.108 [ALERT] ts=14423 TYPE=HUMID_HIGH VALUE=85.0 THRESHOLD=80.0
...
2026-10-21 02:43:09.424 [STATE] RUNNING -> SHUTDOWN
2026-10-21 02:43:09.437 [SUMMARY] alerts TEMP_HIGH=20 HUMID_HIGH=26 TEMP_SPIKE=84 total=130
```

4.2. Số alert quan sát so với kỳ vọng lý thuyết

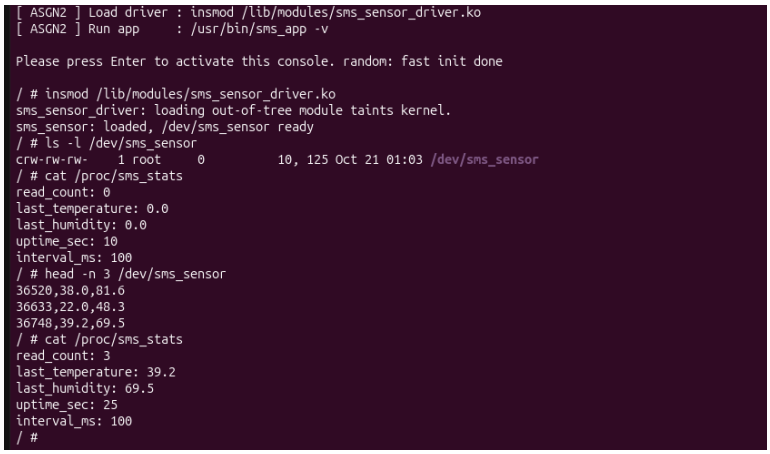
Nguồn dữ liệu là ngẫu nhiên đều (T 15–45 °C, H 30–90 %), nên với 119 mẫu kỳ vọng ≈ 19,8 (T > 40), ≈ 19,8 (H > 80) và ≈ 81,9 (|ΔT| > 5). Quan sát 20 / 26 / 84 nằm trong ±1,5σ: logic phát hiện khớp phân phối nguồn. Số alert cao là hệ quả của dữ liệu mô phỏng, không phải lỗi.



Hình 8 — Alert quan sát (60 s) so với kỳ vọng lý thuyết.

4.3. Tích hợp driver: `/proc/sms_stats`, ioctl, lỗi và file descriptor

Kiểm tra nhanh trên QEMU (trước khi chạy ứng dụng): nạp module, kiểm tra node, đọc `/proc/sms_stats`, đọc 3 mẫu từ `/dev/sms_sensor` — `read_count` tăng 0 → 3 và `last_*` trùng mẫu cuối (39,2 / 69,5).



Hình 9 — `insmod`, `ls -l /dev/sms_sensor` (major 10, minor 125), `cat /proc/sms_stats`, `head -n 3 /dev/sms_sensor`.

Lần chạy ứng dụng 60 s (`proc_sms_stats_*.txt`):

Thời điểm	read_count	last_temp	last_hum	uptime	interval_ms
Trước khi chạy	0	0,0	0,0	0 s	100 (mặc định)
Giữa lần chạy (T ≈ 30 s)	59	42,7	47,5	30 s	450
Sau khi chạy (60 s)	119	44,9	72,1	60 s	450

- **`read_count = 119`** (yêu cầu 120 ± 10), bằng số sample ứng dụng đọc; `last_*` = 44,9 / 72,1 trùng mẫu cuối trong `sms.log`.
- **ioctl**: ứng dụng đặt `interval = -i – guard band` (500 → 450); driver xác nhận qua `interval_ms`: `450` và `dmesg`. Ngoài khoảng cho phép → `-EINVAL`; ứng dụng cũng chặn `-i` ngoài 1–60000 (thoát mã 2).
- **Lỗi `read()`**: `EINTR` → thử lại (trừ khi đang tắt); `EAGAIN` → `usleep(1 ms)` tối đa 50 lần rồi bỏ mẫu + WARN; EOF/parse lỗi → WARN; 10 lỗi liên tiếp → dừng có kiểm soát. Lần chạy chuẩn: `read_errors = 0`.
- **Mã thoát** (host): `-i abc` → **2**; `-D /dev/nonexistent` → **1**; chạy bình thường hoặc SIGINT → **0**.

Bảng file descriptor tại T = 55 s (`fd_table_T55.txt`) chỉ có đúng các fd mong đợi, không có fd mồ côi:

fd	Trỏ tới	Vai trò
0	`/dev/null`	stdin
1, 2	`/tmp/sms_sample_run.txt`	stdout (`-v`) và stderr (`[ALERT]`) được chuyển hướng khi thu log
3	`/tmp/sms.log`	`FILE*` của Logger
4	`/dev/sms_sensor`	fd driver của Sensor
5, 6	`/sms_alarm_queue`	hai descriptor mq: nhận (blocking) và gửi (O_NONBLOCK)

Khi kết thúc, `free_resources()` đóng mọi fd đã mở.

4.4. Memory footprint

Thời điểm	VmSize	VmRSS	Threads
Khởi động (T = 5 s)	4052 kB	1744 kB	5
Ổn định (T = 30 s)	4052 kB	1744 kB	5
Trước khi kết thúc (T = 55 s)	4052 kB	1744 kB	5

RSS không tăng theo thời gian (chênh 0 kB giữa ba thời điểm, cả VmSize lẫn VmRSS). Các lý do: (1) toàn bộ vùng nhớ lớn — hai buffer, mảng jitter 200 KB, stack 256 KB của mỗi thread — được cấp phát *một lần* trước khi thread chạy, không có `malloc/free` trong vòng lặp; (2) không có hàng đợi nào có thể phình vì mọi kênh đều có giới hạn cứng và producer bỏ phần tử khi đầy (không tăng bộ nhớ); (3) log ghi ra file trong `/tmp` (tmpfs) — dung lượng tăng nằm trong page cache của hệ thống chứ không cộng vào RSS của tiến trình, và bị chặn bởi rotate ở 1 MB; (4) `mlockall(MCL_CURRENT|MCL_FUTURE)` khiến các trang được nạp và khóa ngay từ đầu nên RSS đã "phẳng" ngay từ T = 5 s. Số liệu này cũng khớp với kết quả Valgrind (§5.1): toàn bộ 31 khối cấp phát đều được giải phóng.

Phân tích `/proc/PID/maps` (ARM, build tĩnh)

Vùng địa chỉ	Quyền	Kích thước	Giải thích
`00010000–00083000`	r-xp	460 kB	**Text** của `/usr/bin/sms_app`: mã ứng dụng + glibc/pthread liên kết tĩnh (nên không có vùng `.so`)
`0009d000–000a0000`	r--p	12 kB	Dữ liệu chỉ đọc sau khi relocate (RELRO)
`000a0000–000a2000`	rw-p	8 kB	`.data` + `.bss` (biến toàn cục, TLS khởi tạo)
`000a2000–000c6000`	rw-p	144 kB	**[heap]** (brk) của main arena: `sensor_buf`, `log_buf` (256 × 168 B ≈ 42 kB), cấu trúc điều khiển, bộ đệm stdio
`76d7f000–76dbf000`, `76dc0000–…`, `76f3d000–76f7d000`, `76f7e000–…`	rw-p	4 × 256 kB	**Stack của 4 worker thread** (`SMS_APP_STACK_SIZE` = 256 kB). Trước mỗi stack là một trang **guard** 4 kB `---p` (`76d7e000`, `76dbf000`, `76f3c000`, `76f7d000`) — tràn stack sẽ gây SIGSEGV thay vì ghi đè vùng kế bên
`76f7e000–76fef000`	rw-p	452 kB	Stack Sensor (256 kB) liền kề một vùng ~196 kB là mảng `jitter_us[]` 200 000 B (≥ ngưỡng mmap 128 kB nên glibc dùng mmap, không dùng heap)
`76c00000–76d00000`, `76e00000–76f00000`	rw-p + ---p	2 × 1 MB	Hai **heap phụ (arena) của glibc** cho thread không phải main: ~132–136 kB đã commit, phần còn lại dự trữ `---p` (không tốn RAM thật). Kích thước cố định giữa ba lần đo nên không phải leak
`7ed38000–7ed59000`	rw-p	132 kB	**[stack]** của luồng main (tăng dần khi cần)
`[sigpage] [vvar] [vdso] [vectors]`	—	4 kB mỗi vùng	Trang do kernel ARM cung cấp: trampoline tín hiệu, biến thời gian và **vDSO** — chính là "thư viện chia sẻ" của kernel giúp `clock_gettime` không cần vào kernel

Tổng kích thước các vùng trên (không kể `[vectors]`) cộng lại đúng **4052 kB = VmSize**, cho thấy bảng đã giải thích đủ toàn bộ không gian địa chỉ. Nếu liên kết động (như bản `sms_app_x86`), danh sách này sẽ có thêm các vùng `libc.so.6`, `ld.so` với cặp quyền r-xp / r--p / rw-p.

4.5. Đo jitter của Sensor Thread (Task 4)

4.5.1. Phương pháp

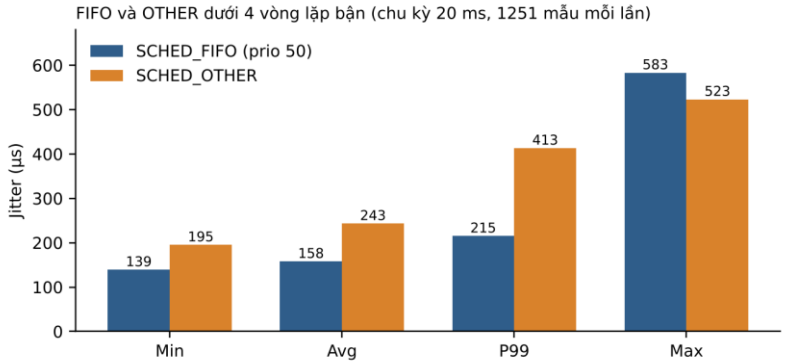
Sensor thread chạy SCHED_FIFO 50 (`pthread_setschedparam` thành công — log ghi `Active policy: SCHED_FIFO (prio 50)`), hẹn giờ bằng `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME)` và gọi `mlockall`

từ đầu. Mỗi chu kỳ ghi `jitter = actual − expected` (µs) vào mảng. So sánh FIFO với OTHER bằng cùng chương trình, cờ `-S fifo|other`, chu kỳ 20 ms, 25 s (1251 mẫu ≥ 1000), dưới tải 4 vòng lặp bận `sh -c 'while :; do :; done'` trên QEMU 2 vCPU và driver thật. P99 tính bằng hạng gần nhất: phần tử thứ [0,99·n] sau khi sắp xếp.

4.5.2. Kết quả so sánh FIFO và OTHER (cấu hình cuối cùng)

Chính sách	Mẫu	Min	Avg	Max	P99
SCHED_FIFO (50)	1251	139 µs	157,6 µs	583 µs	215 µs
SCHED_OTHER	1251	195 µs	243,0 µs	523 µs	413 µs
Cải thiện của FIFO	—	−29 %	−35,1 %	+11 % (xấu hơn)	**−47,9 %**

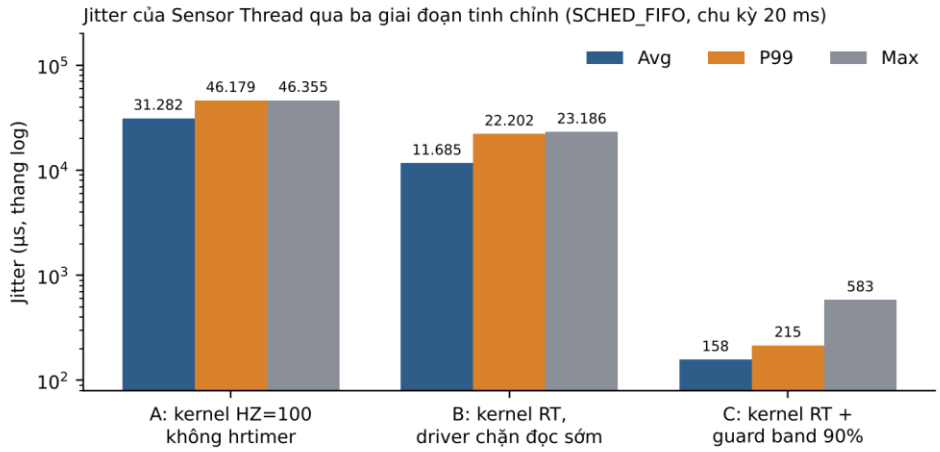
```
SCHED_FIFO  (prio 50)              SCHED_OTHER
0-100 us    :   0 (  0.0%)         0-100 us    :    0 (  0.0%)
100-500 us  : 1250 ( 99.9%)        100-500 us  : 1249 ( 99.8%)
500-1000 us :    1 (  0.1%)        500-1000 us :    2 (  0.2%)
>1 ms       :    0 (  0.0%)        >1 ms       :    0 (  0.0%)
```



Hình 10 — Jitter SCHED_FIFO và SCHED_OTHER dưới tải 4 vòng lặp bận (chu kỳ 20 ms).

Một lần chạy trước đó với cùng chương trình nhưng chỉ 2 vòng lặp bận cho kết quả tương tự về hướng: FIFO **251,7 / 422 / 632 µs** (Avg / P99 / Max) so với OTHER **339,5 / 487 / 667 µs**, tức P99 cải thiện 13,3 %. Hai lần đo cho mức cải thiện P99 từ 13 % đến 48 %: hướng nhất quán (FIFO tốt hơn ở trung bình và đuôi) nhưng độ lớn dao động mạnh, và mỗi cấu hình chỉ được đo một lần nên không thể đưa khoảng tin cậy.

4.5.3. Quá trình tinh chỉnh: ba giai đoạn



Hình 11 — Jitter của Sensor thread (SCHED_FIFO, chu kỳ 20 ms, thang log) qua ba giai đoạn.

Giai đoạn	FIFO: Avg / P99 / Max (µs)	OTHER: Avg / P99 / Max (µs)	Chẩn đoán và hành động

Giai đoạn	FIFO: Avg / P99 / Max (µs)	OTHER: Avg / P99 / Max (µs)	Chẩn đoán và hành động
A. Kernel HZ=100, không hrtimer	31 282 / 46 179 / 46 355	30 793 / 46 161 / 46 332	Cả FIFO và OTHER đều tệ như nhau và ~577 mẫu/25 s thay vì 1251: giới hạn nằm ở độ phân giải bộ hẹn giờ của kernel chứ không ở scheduler. Build lại kernel với `HZ=1000`, `HIGH_RES_TIMERS`, `PREEMPT`
B. Kernel mới, driver chặn read sớm	11 685 / 22 202 / 23 186	11 458 / 22 950 / 23 201	Vẫn không có khác biệt FIFO/OTHER. Bản chạy `-D sim` (không dùng driver) chỉ 649 µs trung bình ⇒ nguyên nhân nằm ở driver: `read()` đến sớm hơn `interval_ms` thì bị `msleep_interruptible`, độ trễ cộng dồn thành răng cưa 0–22 ms
C. Guard band 90 % + kernel RT	157,6 / 215 / 583	243,0 / 413 / 523	Ứng dụng đặt ngưỡng driver = 90 % chu kỳ (18 ms cho 20 ms; 450 ms cho 500 ms) để driver không bao giờ tự chặn; từ đây FIFO mới tách khỏi OTHER

Bài học: chọn SCHED_FIFO không cứu được độ trễ khi nút thắt nằm ở chỗ khác (bộ hẹn giờ kernel, thiết kế driver). Điều này xuất hiện rõ trong các số liệu A và B, nơi hai chính sách cho kết quả gần như trùng nhau. Về cơ chế chính xác trong driver khiến độ trễ biến thành răng cưa 0–22 ms, tôi chưa tách riêng được từng nguyên nhân vi mô; điều đã chứng minh bằng thực nghiệm là: giữ nguyên mọi thứ khác và chỉ hạ ngưỡng của driver xuống 90 % chu kỳ thì trung bình giảm từ 11,7 ms xuống 0,16 ms.

4.5.4. Trả lời các câu hỏi phân tích (§3.9 đề bài)

1. **SCHED_FIFO có cải thiện so với SCHED_OTHER không? Bao nhiêu % ở P99?** Có, ở trung bình và đuôi: P99 giảm **47,9 %** (413 → 215 µs) trong lần đo chuẩn với 4 tác vụ nền, 13,3 % trong lần đo 2 tác vụ nền; trung bình giảm 35 %. Giá trị Max không cải thiện (583 so với 523 µs; hiệu ~60 µs nằm trong dao động của QEMU). Mức cải thiện tuyệt đối khá nhỏ (< 0,2 ms trên chu kỳ 20 ms) vì tải nền là các vòng lặp CPU-bound và bộ lập lịch CFS cho thread vừa thức dậy từ giấc ngủ dài được ưu tiên chen vào; lợi thế của FIFO sẽ lớn hơn khi có nhiều tiến trình cùng tranh chấp thật sự (I/O, softirq) hoặc trên phần cứng thật.
2. **Ngưỡng jitter tối đa chấp nhận được cho cảm biến công nghiệp chu kỳ 500 ms? Có đáp ứng không?** Đây là một giả định thiết kế, không phải hằng số của đề bài: với chu kỳ 500 ms, tôi chọn ngưỡng **≤ 1 % chu kỳ = 5 ms** (đủ để mẫu không bị gán nhầm sang chu kỳ khác và không làm lệch phép tính ΔT/trung bình trượt). Lần chạy 60 s ở 500 ms đo Max = **629 µs** (0,13 % chu kỳ), P99 = 592 µs, tức dư khoảng 8 lần; mọi lần đo ở 20 ms dưới tải cũng ≤ 667 µs. **Đáp ứng.** Đối với bài toán an toàn (ví dụ cần phản ứng < 1 ms) thì kết quả trên QEMU không đủ để kết luận mà phải đo lại trên phần cứng thật.
3. **Muốn giảm jitter thêm nữa cần thay đổi gì?** (a) Kernel **PREEMPT_RT** để các đoạn găng của kernel và threaded IRQ có thể bị preempt; (b) **CPU isolation**: `isolcpus=1 nohz_full=1 rcu_nocbs=1` và `pthread_setaffinity_np` ghim Sensor vào CPU 1, dồn mọi thứ còn lại sang CPU 0; (c) **IRQ affinity**: `/proc/irq/*/smp_affinity` chuyển ngắt khỏi CPU của Sensor; (d) bỏ `printf` khỏi đường nóng (Logger đã tách riêng, nhưng `-v` in từ Processor), dùng mutex `PTHREAD_PRIO_INHERIT`; (e) tắt lọc tần số CPU (governor `performance`); (f) đo nền bằng `cyclictest` để tách phần nhiễu của QEMU/host khỏi phần của ứng dụng.

5. Debug và profiling

Các công cụ dưới đây chạy trên bản x86_64 `sms_app_x86` với bộ mô phỏng cảm biến `-D sim` (host không có `/dev/sms_sensor`); toàn bộ mã đường đi (4 thread, buffer, mq, logger) giống hệt bản ARM, chỉ khác nguồn dữ liệu.

5.1. Valgrind — memcheck

```
valgrind --leak-check=full ./sms_app_x86 -D sim -i 100 -d 35 -l /tmp/sms_vg.log -j
/tmp/jitter_vg.txt

==6814== HEAP SUMMARY:
==6814==     in use at exit: 0 bytes in 0 blocks
==6814==   total heap usage: 31 allocs, 31 frees, 263,316 bytes allocated
==6814== All heap blocks were freed -- no leaks are possible
==6814== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

- **0 bytes definitely lost**, và cũng không có *possibly lost* hay *still reachable* (in use at exit = 0 bytes) nên không cần giải thích thêm; 31 lần cấp phát = 31 lần giải phóng; 0 lỗi (không có đọc/ghi ngoài vùng, không dùng giá trị chưa khởi tạo, không double free).
- Chạy 35,1 s ở chu kỳ 100 ms (349 mẫu đọc), dài hơn mức tối thiểu 30 s. Chương trình kết thúc nhờ hết `-d 35`, đi qua đúng giao thức tắt máy như khi SIGINT (§2.6); đường SIGINT được thử riêng ở §4.3.
- **Giải thích 263 316 B**: mảng jitter 50 000 × 4 B = 200 000 B; `log_queue` 256 × 168 B = 43 008 B; `shared_buffer` 64 × 16 B = 1024 B; phần còn lại (~19 kB) là cấu trúc điều khiển, bộ đệm `FILE*` của glibc và mảng tạm khi sắp xếp P99. Con số không phụ thuộc thời gian chạy ⇒ nhất quán với kết luận "không tăng bộ nhớ" ở §4.4.
- **ThreadSanitizer** (bản `-fsanitize=thread`, 10 s, chu kỳ 20 ms, 500 mẫu): 0 cảnh báo data race / lock-order inversion — kiểm tra riêng khía cạnh đa luồng mà memcheck không bao phủ.

5.2. GDB — Kịch bản A: quan sát `shared_buffer` khi Sensor đang ghi

Chuẩn bị. Bản biên dịch có symbol (`-g`) chạy bằng script GDB (`app/gdb_scenario_a.gdb`). Vì `cbuf_t` là cài đặt chung dùng cho cả `shared_buffer` (64 phần tử) lẫn `log_queue` (256 phần tử), breakpoint đặt có **điều kiện** `cb->capacity == 64` để chỉ bắt đúng buffer cần quan sát:

```
handle SIGUSR1 nostop noprint pass       # SIGUSR1 do main gửi lúc tắt máy, không để GDB dừng
break circular_buffer.c:92  if cb->capacity == 64   # cbuf_push : Sensor đang ghi (trong vùng
găng)
break circular_buffer.c:109 if cb->capacity == 64   # cbuf_pop  : Processor vừa được đánh thức
define ringinfo                                     # in thread hiện tại +
head/tail/size/total_produced
  thread ; print cb->head ; print cb->tail ; print cb->size ; print cb->total_produced
end
run -D sim -i 100 -d 20 -l /tmp/sms_gdb.log -j /tmp/jitter_gdb.txt > /dev/null 2>&1
```

Trạng thái các thread (`info threads` và `thread apply all bt` tại lần dừng đầu tiên, Sensor đang ở `cbuf_push`):

Thread	Vai trò	Đang ở	Call stack rút gọn (nguồn: `gdb_sms.txt`)
1	main	`rt_sigtimedwait`	`__sigtimedwait ← wait_for_end (sms_app.c:293) ← main (sms_app.c:373)`
2	Logger	`pthread_cond_timedwait`	`cbuf_pop_timeout (circular_buffer.c:132) ← logger_thread (logger_thread.c:110)`
3	Alert	`mq_timedreceive`	`alert_thread (alert_thread.c:22)` — chặn chờ cảnh báo

Thread	Vai trò	Đang ở	Call stack rút gọn (nguồn: `gdb_sms.txt`)
4	Processor	`pthread_cond_wait` (futex)	`cbuf_pop (circular_buffer.c:107) ← processor_thread (processor_thread.c:42)`
5	Sensor	**chạy**, dừng ở breakpoint	`cbuf_push (circular_buffer.c:92) ← sensor_thread (sensor_thread.c:188)`

Giá trị quan sát (`ringinfo` tại mỗi lần dừng; `capacity = 64`; thứ tự bản ghi là thứ tự các lần dừng liên tiếp):

#	Dừng tại	head	tail	size	total_produced	Nhận xét
1	Sensor, trước khi tăng head (dòng 92)	0	0	0	0	Buffer rỗng, mẫu đầu tiên sắp vào
2	Processor, vừa ra khỏi `cond_wait` (dòng 109)	1	0	1	1	push đã xong và `signal` đánh thức Processor
3	Sensor, mẫu thứ hai (dòng 92)	1	1	0	1	Processor đã lấy mẫu đầu (`tail` 0→1) trước khi mẫu hai được ghi
4	Processor (dòng 109)	2	1	1	2	Lặp lại đúng chu trình
5	`set scheduler-locking on`; next → dòng 110 (trước `take_locked`)	2	1	1	2	Chỉ Processor chạy, Sensor bị đóng băng
6	next → dòng 113 (sau `take_locked`)	2	2	0	2	Chỉ `tail` và `size` đổi; head, `total_produced` giữ nguyên

Phân tích. Ở cả sáu lần dừng bất biến `size ≡ head − tail (mod 64)` luôn đúng, và `total_produced` chỉ tăng khi `head` tăng, chưa bao giờ nhìn thấy trạng thái dở dang (ví dụ `head` đã tăng mà `size` chưa tăng). Lý do: cả hai breakpoint đều nằm *bên trong* vùng găng giữ `cb->lock` nên khi một thread dừng ở đó, thread kia không thể chạm vào cùng cấu trúc — nó chỉ có thể nằm ở `pthread_mutex_lock` hoặc `cond_wait`. Bước 5–6 với `scheduler-locking on` cho thấy thao tác lấy phần tử (`take_locked`) chỉ thay `tail` và `size`, tách bạch khỏi thao tác ghi (`head`, `total_produced`): đây chính là lý do producer và consumer không giẫm lên nhau. Lưu ý đúng đắn về phương pháp: dừng bằng breakpoint làm thay đổi thời gian, nên việc không quan sát thấy bất thường không tự nó chứng minh không có race; kết luận dựa trên phối hợp với phân tích khóa (§3.2) và ThreadSanitizer (0 cảnh báo).

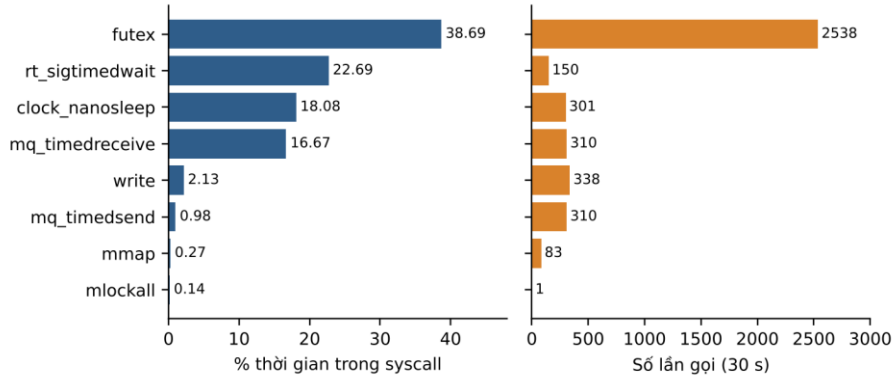
Vấn đề thực tế gặp khi debug. (1) Lần chạy đầu bị dừng vì `SIGUSR1` (tín hiệu main gửi để đánh thức Sensor lúc tắt máy) — khắc phục bằng `handle SIGUSR1 nostop noprint pass`. (2) Breakpoint không điều kiện bắt cả lần push của `log_queue` (cùng hàm `cbuf_push`), nên phải lọc theo `capacity`. Cả hai đều xuất phát từ chính thiết kế của ứng dụng.

5.3. strace — mẫu system call

Chạy `strace -c -f` trong 30 s (bản x86, mô phỏng cảm biến, chu kỳ 100 ms ⇒ ~300 mẫu), kết quả đầy đủ trong `strace_sms.txt`. Năm syscall có % thời gian cao nhất:

#	Syscall	% thời gian	Số lần	Lỗi	Tương ứng với thao tác nào trong ứng dụng
1	`futex`	38,69	2538	162	Nền của `pthread_mutex_lock/unlock` khi có tranh chấp và của `cond_wait/timedwait/signal`: mỗi mẫu đi qua `shared_buffer` (signal Processor), rồi `log_queue` (signal Logger cho SAMPLE và mỗi ALERT), cộng các lần thread ngủ/thức; ≈ 8,5 lệnh futex mỗi mẫu. Mã lỗi nhận được là kết quả bình thường của futex (ví dụ EAGAIN khi giá trị đã đổi trước lúc ngủ), không phải lỗi chức năng

#	Syscall	% thời gian	Số lần	Lỗi	Tương ứng với thao tác nào trong ứng dụng
2	`rt_sigtimedwait`	22,69	150	150	Vòng `sigtimedwait` của main với lát 200 ms: 30 s ÷ 0,2 s = **150** lần, cả 150 lần đều hết thời gian (EAGAIN) vì không có SIGINT — khớp chính xác với code
3	`clock_nanosleep`	18,08	301	1	Sensor ngủ tuyệt đối mỗi chu kỳ: 300 chu kỳ + 1 lần bị SIGUSR1 ngắt (EINTR) khi tắt máy
4	`mq_timedreceive`	16,67	310	0	Alert thread chờ trong `mq_receive` (glibc dùng syscall `mq_timedreceive`): 309 cảnh báo + 1 lính canh ALERT_STOP; bằng đúng số `mq_timedsend` (310) ⇒ mọi thông điệp gửi đi đều được nhận
5	`write`	2,13	338	0	≈ 309 lần Alert ghi `[ALERT]` ra stderr (không đệm), cộng các lần Logger xả file mỗi 5 s (đệm stdio 4 kB) và dòng tổng kết



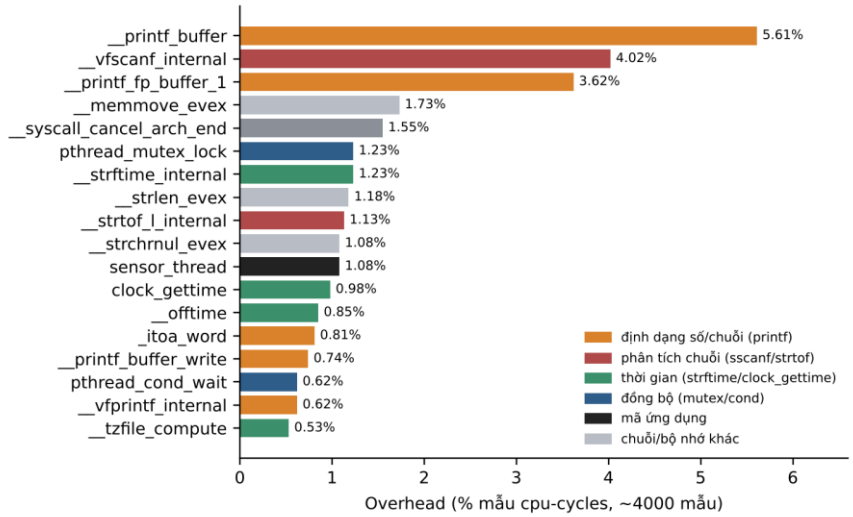
Hình 12 — Top syscall theo % thời gian và theo số lần gọi (30 s).

Một chi tiết đáng chú ý: `mmap` gọi 83 lần nhưng 56 lần trả lỗi. Trên host, tiến trình chạy với người dùng thường nên `mlockall(MCL_FUTURE)` cùng giới hạn `RLIMIT_MEMLOCK` khiến các `mmap` cấp vùng lớn cho heap phụ của glibc (dự trữ 64 MB) trả `EAGAIN` (man 2 mmap: "too much memory has been locked"); glibc xử lý bằng cách quay về arena chính, ứng dụng vẫn chạy bình thường (Valgrind không báo lỗi). Trên QEMU tiến trình chạy bằng root nên không bị giới hạn này. Đây là chi phí cần biết khi dùng `mlockall` trên tài khoản thường.

User-mode hay kernel-mode, CPU-bound hay I/O-bound? `/usr/bin/time -v` cho lần chạy 30,03 s: user **0,04 s**, system **0,04 s**, CPU **0 %**, 1439 lần chuyển ngữ cảnh tự nguyện, chỉ 2 lần không tự nguyện, Max RSS 3852 kB. Tổng thời gian CPU (0,08 s) chiếm khoảng 0,27 % thời gian thực: ứng dụng ngủ gần như toàn bộ (chờ hẹn giờ, mq, futex). Tỷ lệ user:kernel xấp xỉ 1:1 cho thấy phần lớn chi phí CPU ít ỏi còn lại là gọi hệ thống và đồng bộ, không phải tính toán. Kết luận: **thiên về I/O-bound / chờ sự kiện (event-driven), không CPU-bound**. Thời gian trong bảng strace thuộc về các lời gọi *chặn* (futex, sigtimedwait, nanosleep, mq_receive) — đó là thời gian ngủ, không phải thời gian tiêu CPU.

5.4. perf — điểm nóng CPU

Ở chu kỳ mặc định 100 ms, `perf record` chỉ thu được **7 mẫu** trong cả lần chạy — một bằng chứng nữa là ứng dụng gần như rảnh. Để có thống kê, tôi chạy lại với chu kỳ 10 ms và tần số lấy mẫu `-F 4999` (và `kernel.kptr_restrict=0` để thấy địa chỉ kernel): `sudo perf record -g -F 4999 ./sms_app_x86 -D sim -i 10 …` → ~4000 mẫu `cpu-cycles` (`perf_sms.txt`; bản 100 ms giữ ở `perf_sms_100ms.txt`).



Hình 13 — Các hàm tốn nhiều CPU nhất (perf report, ~4000 mẫu, chu kỳ 10 ms).

Nhóm	Overhead	Thành phần
Định dạng chuỗi/số (printf)	≈ 11,4 %	`__printf_buffer` 5,61 · `__printf_fp_buffer_1` 3,62 · `_itoa_word` 0,81 · `__printf_buffer_write` 0,74 · `__vfprintf_internal` 0,62 (chưa tính `hack_digit` 0,56 và `__mpn_divrem` 0,53 — cũng là in số thực)
Phân tích chuỗi (scanf/strtof)	≈ 5,2 %	`__vfscanf_internal` 4,02 · `__strtof_l_internal` 1,13
Thời gian (timestamp log)	≈ 3,6 %	`__strftime_internal` 1,23 · `clock_gettime` 0,98 · `__offtime` 0,85 · `__tzfile_compute` 0,53
Đồng bộ	≈ 2,9 %	`pthread_mutex_lock` 1,23 · `cond_wait` 0,62 · `cond_timedwait` 0,56 · `cond_signal` 0,51
Mã ứng dụng (`sensor_thread`)	1,08 %	Toàn bộ logic của ứng dụng ngoài thư viện

Có bất ngờ không? Có, hai điều. (1) *Logic nghiệp vụ gần như không tốn CPU* (thread Sensor chỉ 1,08 %); điểm nóng toàn là thư viện chuẩn xoay quanh **chuyển đổi văn bản ↔ số**. (2) `sscanf` + `strtof` để parse một dòng CSV chỉ vài chục byte tốn nhiều hơn cả mutex/condvar (5,2 % so với 2,9 %), và việc tạo timestamp bằng `localtime_r/strftime` cho *mỗi dòng log* (kéo theo `__tzfile_compute`, `__offtime`) cũng đáng kể. Lý do cấu trúc: dữ liệu đi vòng "số → chuỗi (driver) → số (Sensor) → chuỗi (log)".

Đề xuất cải tiến (có cơ sở từ số liệu):

1. **Parser số cố định thay `sscanf/strtof`**: driver luôn xuất dạng `dd.d` (nhiệt độ/độ ẩm ×10) nên có thể viết hàm đọc số nguyên rồi chia 10 — loại bỏ ≈ 5,2 % mẫu CPU và các lệnh gọi locale.
2. **Lưu nhiệt độ/độ ẩm dạng số nguyên ×10 và in bằng hàm tự viết** thay cho `%f` trong `logger_post` (Processor): bỏ qua đường `__printf_fp` nặng (≈ 4–5 % gồm cả `hack_digit`, `__mpn_divrem`).
3. **Cache chuỗi thời gian theo giây** trong Logger (chỉ gọi `localtime_r/strftime` khi giây thay đổi, phần mili-giây ghép bằng số nguyên): bỏ ≈ 2,6 % (`strftime`, `offtime`, `tzfile`).
4. Dài hạn: nếu tần suất mẫu tăng, ghi log nhị phân (cấu trúc cố định) và giải mã ngoài thiết bị — loại bỏ toàn bộ nhóm printf khỏi thiết bị nhúng.

Với tải hiện tại (CPU ≈ 0 %) các cải tiến này không cần thiết để chạy đúng, nhưng chúng có ý nghĩa với thiết bị nhúng chạy pin hoặc khi tăng tần suất lấy mẫu; số liệu perf cho thấy chúng nhắm đúng các hàm tốn CPU nhất.

6. Reflection — nhìn lại toàn khóa

6.1. Kỹ năng khó nhất và cách tôi vượt qua

Khó nhất với tôi không phải viết thêm một thread hay một cấu trúc dữ liệu, mà là **suy luận về thời gian qua nhiều tầng** — tìm xem độ trễ thật sự nằm ở đâu. Khi bắt đầu đo jitter của Sensor thread, tôi tin chắc rằng đặt `SCHED_FIFO` là đủ. Số đo đầu tiên là trung bình **31 ms** trên chu kỳ 20 ms, và điều gây bối rối là FIFO và OTHER cho kết quả gần như giống hệt nhau (31 282 so với 30 793 µs). Chính sự giống nhau đó là manh mối: nếu chính sách lập lịch không tạo ra khác biệt thì nút thắt không nằm ở lập lịch. Tôi lần lượt gỡ từng tầng: build lại kernel với `HZ=1000`, hrtimer và `PREEMPT` (31 ms → 11,7 ms), rồi chạy bản `-D sim` làm thí nghiệm đối chứng (không đi qua driver, chỉ 0,65 ms) để chứng minh phần 11 ms còn lại nằm ở driver chặn `read()` đến sớm; cuối cùng đặt ngưỡng driver bằng 90 % chu kỳ để được 0,16 ms. Điều tôi mang theo: *đo trước, đoán sau; mỗi lần chỉ đổi một biến; luôn có một thí nghiệm đối chứng*.

Khó thứ hai là **debug đa luồng bằng GDB**. Tôi vấp hai chỗ: `cbuf_t` là cài đặt chung nên breakpoint trong `cbuf_push` sẽ bắt cả `log_queue`, và lần chạy đầu bị GDB dừng bởi chính tín hiệu `SIGUSR1` của ứng dụng lúc tắt máy. Giải pháp là breakpoint có điều kiện theo `capacity`, `handle SIGUSR1 nostop noprint pass` và `set scheduler-locking on` để chỉ một thread chạy khi `next`. Sau đó tôi mới thấy giá trị thực của việc nắm vững mô hình: khi nhìn thấy `head/tail/size` thay đổi từng bước, bất biến của buffer trở nên cụ thể thay vì chỉ là lý thuyết.

7. Tài liệu tham khảo và công cụ

1. Christopher Hallinan, *Embedded Linux Primer* (2nd ed.), chương 8–19.
2. Corbet, Rubini, Kroah-Hartman, *Linux Device Drivers* (3rd ed.) — character driver, ioctl, procfs/seq_file.
3. Michael Kerrisk, *The Linux Programming Interface* — chương về POSIX threads, tín hiệu, POSIX message queue.
4. David Butenhof, *Programming with POSIX Threads* — condition variable, thứ tự khóa, tắt thread an toàn.
5. Man pages: `pthread_create(3)`, `pthread_cond_wait(3)`, `mq_open(3)`, `mq_receive(3)`, `clock_nanosleep(2)`, `sched_setscheduler(2)`, `mlockall(2)`, `sigtimedwait(2)`, `mmap(2)`.
6. Brendan Gregg, *Systems Performance* (2nd ed.) và trang brendangregg.com/perf.html — `perf`, phân tích CPU.
7. Tài liệu kernel: `Documentation/timers/hrtimers`; PREEMPT_RT — wiki.linuxfoundation.org/realtime/.
8. Valgrind User Manual (memcheck); GDB manual — *Debugging Programs with Multiple Threads*; tài liệu ThreadSanitizer của GCC.
9. Mã nguồn và ghi chép LAB-01/02/03 và ASSIGNMENT-01 của chính tôi (build kernel 5.15, initramfs BusyBox, module kernel cơ bản).

Phụ lục

A. Cấu trúc gói nộp

```
SE202007_NguyenNgocQuang_Asgn02/
├── driver/   sms_sensor_driver.c  sms_sensor.h  Makefile  sms_sensor_driver.ko
├── app/      sms_app.c  sensor_thread.[ch]  processor_thread.[ch]  alert_thread.[ch]
│             logger_thread.[ch]  circular_buffer.[ch]  sms_common.[ch]  Makefile
│             sms_app (ARM, static)  sms_app_x86 (x86_64)  gdb_scenario_a.gdb
├── output/   initramfs_asgn2.cpio.gz
├── logs/     sms_sample_run.txt  sms.log  valgrind_sms.txt  strace_sms.txt  perf_sms.txt
│             gdb_sms.txt  proc_sms_stats_{1_before,2_during,3_after}.txt  jitter_report.txt
│             memory_T{5,30,55}.txt  proc_maps.txt  fd_table_T55.txt  time_sms.txt  ...
├── demo/     SE202007_demo.mp4
└── bao_cao/  SE202007_NguyenNgocQuang_Asgn02_BaoCao.pdf
```

B. Lệnh tái hiện

```
# build (không có warning với -Wall -Wextra)
cd app && make host          # sms_app_x86  (x86_64, dùng cho Valgrind/perf/GDB)
cd app && make arm           # sms_app      (ARMv7, static, cho initramfs)
cd driver && make            # sms_sensor_driver.ko  (KDIR = kernel 5.15 đã build)

# trên QEMU (kernel zImage_asgn2_rt + output/initramfs_asgn2.cpio.gz)
insmod /lib/modules/sms_sensor_driver.ko ; cat /proc/sms_stats
/usr/bin/sms_app -v -d 60                    # chạy chuẩn 60 s, Ctrl+C để dừng sớm
/usr/bin/sms_app -i 20 -d 25 -S fifo  -j /tmp/jitter_report.txt   # jitter FIFO
/usr/bin/sms_app -i 20 -d 25 -S other -j /tmp/jitter_report.txt   # jitter OTHER

# trên host (không cần driver: -D sim)
valgrind --leak-check=full ./sms_app_x86 -D sim -i 100 -d 35
strace -c -f ./sms_app_x86 -D sim -i 100 -d 30
sudo perf record -g -F 4999 ./sms_app_x86 -D sim -i 10 -d 20 && sudo perf report --stdio
gdb -x gdb_scenario_a.gdb ./sms_app_x86_dbg
```

C. Trích đoạn mã chính

Hẹn giờ tuyệt đối và đo jitter (`sensor_thread.c`):

```
sms_timespec_add_ms(&next, ctx->cfg.interval_ms);
do { rc = clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next, NULL); }
while (rc == EINTR && atomic_load(&ctx->running));
clock_gettime(CLOCK_MONOTONIC, &actual);
jitter_us = sms_timespec_diff_us(&actual, &next);        /* thực tế - dự kiến */
if (ctx->jitter_count < SMS_APP_JITTER_MAX) ctx->jitter_us[ctx->jitter_count++] =
(int32_t)jitter_us;
```

Guard band cho driver (`sms_app.c`):

```
unsigned int margin = interval / SMS_APP_DRV_GUARD_DIV;      /* 10 % chu kỳ */
interval -= (margin > 0u) ? margin : 1u;                     /* 500 ms -> 450 ms */
ioctl(ctx->dev_fd, SMS_SET_INTERVAL, &interval);
```

Consumer của circular buffer (`circular_buffer.c`) — vòng `while` chống spurious wake-up, trả phần tử còn lại trước khi báo đóng:

```
pthread_mutex_lock(&cb->lock);
while (cb->size == 0 && !cb->closed)
    pthread_cond_wait(&cb->not_empty, &cb->lock);
if (cb->size > 0) take_locked(cb, item); else rc = CBUF_CLOSED;
pthread_mutex_unlock(&cb->lock);
```